2026-11-06 로그인 복귀, 검색, 알림 SSE

- 날짜: 2026-11-06
- 앱: `lgcns-fe-next`
- 이 문서는 오늘 디스크에 있는 코드만 적는다. 루트 `app/layout.tsx`에는 이 기능용 `<script>`가 없다.

세 가지는 서로 다른 시각에 실행된다. 검색 폼은 브라우저의 기본 GET 제출이다. 로그인 복귀 쿠키는 서버가 `Set-Cookie`로 심는다. 알림 SSE는 hydrate 이후 `EventSource`가 연다.

1. 선언과 로딩 시각

파일	선언	실행 시점	async / defer / type="module"과의 차이
`component/molecules/PostSearchForm.tsx`	`export default function PostSearchForm`. `"use client"` 없음	서버가 폼 HTML을 만든다. 제출은 브라우저가 `GET /posts?q=&sort=`로 이동한다	스크립트가 아니다. 제출은 HTML 폼의 기본 동작이다
`app/(post)/posts/page.tsx`	`async function PostsContent`. `searchParams`는 `Suspense` 자식에서 `await`	요청마다 서버가 `q`, `page`, `sort`를 읽은 뒤 `getCachedPosts`를 호출한다	함수의 `async`는 ECMAScript 비동기 함수다. `<script async>`가 아니다
`lib/postCache.ts`	`export async function getCachedPosts`. 본문 첫 줄 `"use cache"`	서버 렌더 중 호출된다. 인자가 같으면 캐시된 목록을 반환한다	`"use cache"`는 번들러 지시문이다

파일	선언	실행 시점	async / defer / type="module"과의 차이
actions/returnToActions.ts	파일 첫 줄 "use server". export async function rememberReturnPath	로그인 대화상자나 헤더의 로그인 버튼이 호출할 때 서버에서 쿠키를 쓰고 /login으로 보낸다	"use server"는 <script>의 async가 아니다
app/auth/continue/route.ts	export async function GET	로그인 공급자가 callbackUrl인 /auth/continue로 보낸 뒤, 그 GET이 쿠키를 읽고 지운 다음 리다이렉트한다	라우트 핸들러다. 브라우저 모듈 그래프에 없다
proxy.ts	export async function proxy	매칭된 문서 요청마다 서버가 세션을 확인하고, 필요하면 응답에 Set-Cookie를 붙인다	페이지의 <script>보다 먼저, 응답을 만들기 전에 실행된다
component/molecules/NotificationStream.tsx	파일 첫 줄 "use client". useEffect 안에서 new EventSource	서버 HTML이 그려진 뒤 브라우저가 클라이언트 번들을 받아 hydrate하고, 탭이 보일 때 연결한다	EventSource는 DOM API다. 이 파일 경로를 src로 받는 <script type="module">은 없다
app/api/notifications/stream/route.ts	export async function GET. await connection()	위 EventSource의 GET이 살아 있는 동안 text/event-stream 본문을 이어서 쓴다	응답 본문이 이벤트 스트림이다. defer 스크립트가 아니다

파일	선언	실행 시점	async / defer / type="module"과의 차이
service/notificationWatch.ts	function subscribe, async function connect	SSE 구독이 있을 때 서버 프로세스에서 MongoDB change stream 하나를 연다	Node에서 실행된다. 브라우저가 이 파일을 내려받지 않는다
app/api/auth/handoff/route.ts	export async function GET. 응답 HTML 안의 <script>	로컬로 되돌릴 주소가 있을 때만 HTML을 반환한다. 그 스크립트는 async도 defer도 type="module"도 없다	파서가 그 태그를 만나는 시점의 고전 스크립트다. location.replace를 즉시 실행한다

HTML에서 async 스크립트는 받는 즉시 실행하고 문서 파싱을 막지 않으며 순서가 없다. defer는 파싱을 막지 않고, 문서 파싱이 끝난 뒤 적힌 순서대로 실행한다. type="module"은 그 지연 실행에 import/export가 더해진다. 검색, 복귀 쿠키, 알림 스트림은 그 세 속성으로 붙지 않는다.

참고: HTML script 요소, MDN script, ECMA-262 비동기 함수, HTML 서버 전송 이벤트, MDN EventSource.

2. 회원만 쓸 수 있는 기능과 복귀

좋아요, 댓글, 글 삭제, 글쓰기, /cart, /order는 세션이 있을 때만 서버가 저장한다. 없을 때는 가려는 주소를 returnTo 쿠키에 남기고 /login으로 보낸다. 로그인이 끝나면 /auth/continue가 그 주소로 다시 보낸다.

2.1 좋아요를 눌렀을 때

PostCard는 "use client"다. useAuth의 isLoggedIn이 거짓이면 allowLike가 거짓을 반환하고 LoginRequiredDialog를 연다. 대화상자 문구는 로그인이 필요해요, 설명은 좋아요는 로그인 후 이용할 수 있어요.다. 로그인 버튼은 rememberReturnPath에 returnTo로 pathname + search를 넘긴다.

PostLikeForm도 "use client"다. onBeforeSubmit이 거짓이면 useOptimistic을 호출하지 않고 서버 액션도 호출하지 않는다. 회원이면 returnTo를 FormData에 넣고 setLike(!like.liked)로 개수와 눌림을 먼저 바꾼 뒤 toggleLike을 기다린다. applyLike는 같은 상태면 그대로 두고, 아니면 1을 더하거나 빼며 0 아래로 내리지 않는다.

toggleLike, createComment, deletePost, createPost, updatePost는 "use server"인 actions/postActions.ts의 author를 탄다. readCurrentUser가 없으면 FormData의 returnTo를 saveReturnTo로 저장하고 redirect("/login")한다. 값이 비어 있으면 각 액션이 넘긴 대체 경로를 쓴다. 좋아요의 대체 경로는 글 id가 ObjectId면 /postdetail/[id], 아니면 /posts다.

댓글 폼과 삭제 폼도 제출 전에 같은 returnTo를 넣는다. 삭제는 window.confirm("이 글을 삭제할까요?")가 거짓이면 제출을 막는다.

헤더의 LoginStatus는 비로그인일 때 같은 액션으로 현재 주소를 저장한다. 로그아웃은 signOut({ callbackUrl: "/" })이다.

2.2 쿠키

이름 returnTo. 옵션은 httpOnly, sameSite: "lax", path: "/", maxAge 30분, secure는 NODE_ENV === "production"일 때만 참이다. 값은 safeCallbackPath를 거친 뒤 encodeURIComponent한다.

safeCallbackPath는 빈 값, /로 시작하지 않는 값, //로 시작하는 값, /login, /signup, /auth/continue를 /로 바꾼다. 쿼리는 경로만 검사하고, 원래 문자열에 쿼리가 있으면 그 문자열을 유지한다.

심는 곳:

- proxy의 loginRedirect. 비회원이 /postwrite, /cart, /order로 오면 요청 경로와 쿼리를 쿠키에 넣고 /login으로 보낸다.
- 서버 액션 rememberReturnPath, author.
- app/(post)/(postsys)/postwrite/page.tsx의 PostWriteContent. 세션이 없으면 saveReturnTo(writePath(postId)) 다음 redirect("/login")한다.

지우는 곳:

- app/auth/continue/route.ts의 GET. jar.delete(RETURN_TO_COOKIE) 후 그 경로로 NextResponse.redirect.
- proxy의 continueRedirect. 이미 로그인한 사용자가 /login 또는 /signup에 있으면 쿠키를 읽고 maxAge: 0으로 비운 뒤 그 경로로 보낸다.

httpOnly라 document.cookie로는 이 값을 읽지 못한다.

참고: MDN Set-Cookie, Next.js cookies, Next.js redirect.

2.3 콜백 주소

CONTINUE_PATH는 /auth/continue다. LoginForm, KakaoLoginButton, GoogleLoginButton의 signIn은 redirect: true, callbackUrl: CONTINUE_PATH다. 자격 증명 로그인은 email 필드에 아이디를 넣는다. 공급자가 성공하면 NextAuth가 /auth/continue로 보내고, 그 라우트가 returnTo 쿠키의 주소로 한 번 더 보낸다. 콜백 주소 자체는 글 주소가 아니다.

app/api/auth/handoff/route.ts는 다른 경로다. 쿼리 returnTo가 localhost 또는 127.0.0.1의 http/https가 아니면 400이다. 세션의 accessToken이 없으면 /login?callbackUrl=에 이 핸드오프 경로와 쿼리를 붙여 보낸다. 있으면 목적지 해시에 accessToken=을 넣고, location.replace 스크립트가 있는 HTML을 반환한다.

3. 검색

PostSearchForm의 action은 /posts다. method를 적지 않았으므로 GET이다. 숨은 필드 sort는 현재 정렬을 유지하고, 입력 이름 q가 키워드다. maxLength는 100이다. 키워드가 있으면 지우기 링크가 postsListHref(1, sortOrder, "")로 간다.

PostsContent는 Suspense 안에서 searchParams를 기다린다. q는 normalizePostKeyword를 거친다. 구현은 postService.normalizeKeyword다. 앞뒤 공백을 제거하고, 연속 공백을 하나로 줄이고, 100자를 넘으면 자른다.

listPosts는 키워드가 있으면 authorName 또는 body에 contains와 mode: "insensitive"를 건다. 페이지 크기는 20이다. 정렬은 createdAt의 asc/desc다. 개수가 0이면 글 배열은 빈 배열이다. 페이지 번호는 전체 페이지 안으로 자른다.

getCachedPosts(pageNumber, sortOrder, keyword)는 "use cache", cacheTag("posts"), cacheLife("max")다. 키워드가 함수 인자이므로 다른 q는 다른 캐시 항목이다. 목록 캐시가 비면 console.log에 q=가 찍힌다. GET /api/posts도 같은 page, sort, q로 getCachedPosts를 호출하고, 세션이 있으면 likedPostIds만 캐시 밖에서 붙인다.

페이지네이션과 최신순/오래된순 링크는 postsListHref로 page, sort, q를 함께 유지한다. 검색 결과가 없으면 PostListSection은 키워드에 맞는 글이 없다는 문장을 보여 준다.

참고: HTML 폼 제출, MDN form, Next.js use cache.

4. 알림과 SSE

다른 사람이 내 글에 좋아요를 누르거나 댓글을 달면 Notification 행이 생긴다. 내가 한 행동은 notifyAuthor가 authorId === actorId이면 만들지 않는다. 좋아요를 취소하면 그 사용자의 kind: "like" 알림을 지운다. 글을 지우면 그 postId의 알림을 지운다.

4.1 처음 그릴 때

app/(main)/layout.tsx와 app/(post)/layout.tsx는 AppHeader를 둔다. HeaderData는 readNotificationSnapshot을 기다린다. 세션이 없으면 null이고, 있으면 listForUser가 createdAt 내림차순 20개와 readAt === null인 개수를 반환한다. 이 조회는 Suspense 안에 있다. fallback 헤더는 notifications={null}이라 메뉴를 그리지 않는다.

markNotificationRead와 markAllNotificationsRead는 서버 액션이다. 템플릿 Header는 이 함수를 props로 받아 클라이언트 organism NotificationMenu에 넘긴다. 메뉴와 스트림 molecule은 @/service를 import 하지 않는다.

항목을 열면 FormData에 notificationId와 postId를 넣고 markNotificationRead를 호출한다. 성공하고 postId가 24자리 hex이면 redirect("/postdetail/" + postId)한다. 모두 읽음은 readAt이 비어 있는 행을 지금 시각으로 갱신한다.

4.2 SSE가 하는 일

NotificationStream은 탭이 visible일 때만 new EventSource("/api/notifications/stream")을 연다. 숨으면 close하고, 다시 보이면 새로 연다. 언마운트할 때도 닫는다. onmessage의 data를 JSON.parse해서 onEvent로 넘긴다.

라우트는 connection() 다음 readCurrentUser를 본다. 사용자가 없으면 401이고 본문이 없다. 있으면 Content-Type: text/event-stream인 ReadableStream을 반환한다. 먼저 retry: 10000을 보낸다. 이는 브라우저가 끊긴 뒤 다시 연결하기 전에 기다리는 밀리초다. 그다음 프로세스의 change stream 구독을 등록하고, 스트림이 준비되면 listForUser 결과로 type: "snapshot"을 보낸다. 스냅샷 전에 도착한 변경은 배열에 모아 두었다가 스냅샷 다음에 보낸다. 25초마다 : ping 주석을 보내 연결을 유지한다. 주석은 EventSource가 이벤트로 올리지 않는다. 요청이 취소되면 구독을 해제한다.

메뉴는 스냅샷으로 목록과 배지를 바꾸고, 이미 본 적 없는 안 읽은 항목만 토스트로 띄운다. 토스트는 최대 3개이고 4초마다 가장 오래된 하나를 뺀다. insert는 목록 앞에 넣고 20개만 남긴다. read는 그 id를 읽음으로 바꾼다. delete는 목록에 있는 항목만 빼고, 안 읽음이었으면 배지를 1 줄인다. 로그인 상태가 바뀌면 렌더 중에 목록, 토스트, 패널을 비운다. seenRef를 비우는 일은 effect에서만 한다.

4.3 change stream과의 관계

service/notificationWatch.ts가 MongoDB 드라이버로 컬렉션 Notification을 watch한다. 구독자가 있는 동안 프로세스당 커서 하나다. SSE 연결마다 커서를 만들지 않고, recipientId가 같은 리스너에게만 이벤트를 넘긴다.

변경	SSE type	메뉴가 하는 일
insert	insert와 item	목록 앞에 추가. 안 읽음이고 처음 보는 id면 토스트
update의 readAt	read와 id	그 항목을 읽음으로 표시
readAt이 있는 replace	read	위와 같다
delete	delete와 id	목록에 있으면 제거

삭제의 수신자는 fullDocumentBeforeChange.recipientId다. 시작할 때 collMod로 changeStreamPreAndPostImages를 켜려고 한다. 그 명령이 실패하면 플래그를 올리지 않고, 삭제 이벤트

에 수신자가 없으면 그 삭제는 보내지 않는다. 스트림이 끊기면 구독자가 남아 있을 때 5초 뒤 다시 `watch`한다.

이 커서는 레플리카 셋에서만 동작한다. 단독 MongoDB에서는 change stream이 열리지 않는다. 그때도 첫 화면의 서버 렌더 목록은 남아 있고, 서버 로그에는 `알림 변경 스트림을 열지 못했습니다.`가 한 번 남는다.

참고: MongoDB change streams, MDN 서버 전송 이벤트, Next.js 라우트 핸들러, React useOptimistic.

5. 이 코드가 하지 않는 일

- 알림을 WebSocket이나 서비스 워커 푸시로 보내지 않는다. `ServiceWorkerRegister`는 프로덕션에서 `/sw.js`만 등록한다.
- 알림 목록을 5초마다 서버 액션으로 다시 읽지 않는다. 주기적인 서버 작업은 SSE의 `: ping`과, 스트림이 끊겼을 때의 change stream 재시작이다.
- 검색은 댓글 본문을 찾지 않는다. `authorName`과 글 `body`만 `contains`한다.
- `returnTo`를 `localStorage`에 두지 않는다.